

Dart 1: Expresiones, variables y constantes

Contenidos

- Dart 1: Expresiones, variables y constantes
 - El lenguaje Dart
 - Comentarios
 - Sentencias y expresiones
 - Sentencias
 - Expresiones
 - Operadores aritméticos
 - Funciones matemáticas
 - Variables
 - Nombres de variable con sentido
 - Inferencia de tipos
 - const y final
 - Incrementar y decrementar
 - Mini ejercicios

El lenguaje Dart

Cuando se escriben aplicaciones **Flutter**, se usa el lenguaje de programación **Dart**. Por tanto, para poder escribir buenas aplicaciones Flutter, es necesario conocer el lenguaje Dart.

Nota: Dart es un lenguaje de programación desarrollado por Google recientemente. Su primera versión se publicó en 2013.

Una buena forma de aprender el lenguaje Dart, sin tener que instalar ninguna herramienta en el ordenador es usar **Dartpad**. DartPad es una herramienta online que permite explorar el lenguaje y escribir programas en Dart mediante un navegador web normal.



Para acceder a DartPad solo hay que teclear esta dirección <https://dartpad.dev/> en cualquier navegador web. En la imagen de la página anterior puede verse el aspecto de DartPad en el navegador **Safari** de MacOS.

La ventana está dividida en tres zonas:

- **Editor:** Se encuentra ocupando la parte izquierda de la ventana. Es la parte en la que se escribe el código Dart que se quiera probar.
- **Consola:** La consola de depuración (arriba a la derecha) es donde se muestran los mensajes que se producen al ejecutar el código del editor.
- **Documentación:** Abajo y a la derecha se muestra la zona de documentación. Para todas aquellas APIs documentadas, si se hace clic sobre algún elemento que tenga documentación (una clase, un método, etc.) su documentación se muestra en esta zona.

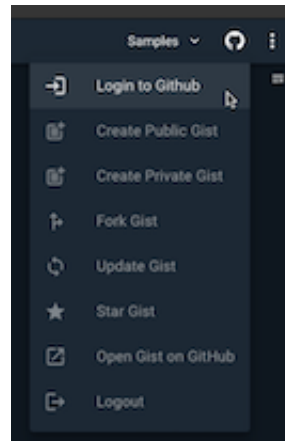
La ventana también tiene un botón etiquetado como **Run** que compila y ejecuta el código. El resultado de cualquier ejecución puede verse en la consola.

Se pueden probar diferentes cosas en DartPad de forma sencilla si se tiene una cuenta de GitHub, ya que DartPad permite usar gits.

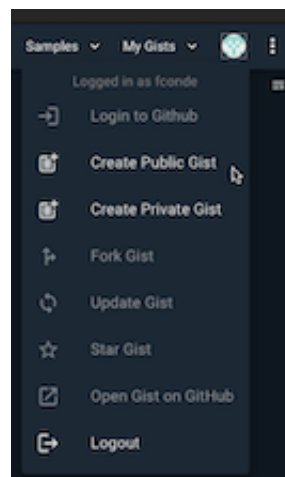
Nota: Podemos pensar en un gits como en una versión light de un repositorio. Permite compartir código de forma muy sencilla usando GitHub.

Muchos programadores lo usan como portapapeles.

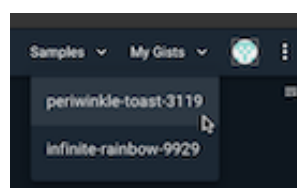
Para usar gits en DarPad, primero hay que iniciar sesión en GitHub. Para ello se usa el menú: [Login to GitHub](#) del menú GitHub (situado en la parte superior derecha de la ventana de DartPad).



Al hacerlo, el menú de GitHub cambia y aparecen nuevas opciones. Usaremos [Create Public Gits](#) para crear nuevos gits en los que probar otras partes de las prácticas (también se pueden crear gits privados)



Una vez que se ha iniciado sesión en GitHub, en la parte superior derecha de la ventana de DartPad aparece un nuevo menú llamado [My Gits](#) v que se usa para cambiar entre dos gits creados previamente.



Comentarios

Son texto escrito junto al código que es ignorado por el compilador

```
// Este es un tipo de comentario en Dart
```

En Dart también se pueden escribir comentarios de este tipo:

```
/* Esto es un comentario que se extiende varias  
líneas */
```

Dart también permite **comentarios anidados**

```
/* Primer nivel de comentarios  
/* Segundo nivel dentro del anterior  
   Otra línea dentro del nivel interior */  
Segunda línea dentro del primer nivel */
```

En Dart hay un tercer tipo de comentarios llamados **comentarios de documentación** que se usan para documentar cualquier variable, función, método o clase pública. Se pueden escribir de dos formas:

```
/// Soy un comentario de documentación  
/// que se extiende a lo largo de dos líneas  
  
/** Yo también  
    soy un comentario en dos líneas */
```

Los comentarios de documentación se escriben justo encima del elemento que se quiera documentar. Por ejemplo:

```
/// Este es un comentario a la función `main`  
void main() {  
    print('Hola mundo!');  
}
```

Una ventaja de usar comentarios de documentación es que, en la mayoría de IDEs, se puede acceder con facilidad a estos comentarios.

En el caso de DartPad, haciendo clic en un elemento documentado, su documentación se muestra en la zona de la pantalla titulada documentación (abajo a la derecha).

Nota: Los comentarios de documentación soportan el formato **Markdown**.

Muy importante: Nunca usar comentarios para explicar cómo funciona un código poco claro. En su lugar reescribir el código para que se entienda por sí solo.

Sentencias y expresiones

Sentencias

Una **sentencia** es una orden, algo que le dice al ordenador lo que hacer.

Importante: Todas las **sentencias simples** terminan con un punto y coma.

Por ejemplo:

```
print(`Hola mundo!`);
```

es una sentencia que imprime la cadena de caracteres `hola mundo` en la consola de depuración (debug console).

Las **sentencias complejas** y los **bloques de código** que usan llaves no necesitan el punto y coma después de las llaves. Por ejemplo:

```
if (someCondition) {  
    // bloque de código  
}
```

Expresiones

Una **expresión** es un valor o algo que pueda ser calculado como un valor. Ejemplos de expresiones en Dart son:

```
42  
3 + 2  
'Hola Mundo!'  
x
```

Operadores aritméticos

En Dart también están los operadores aritméticos básicos: `+` `-` `*` `/` Por ejemplo, la expresión:

```
22 / 7
```

se evalúa como:

```
3.142857142857143
```

Atención: Si se quiere probar la expresión anterior en DartPad (o cualquier otra expresión o grupo de sentencias), hay que escribir un programa completo que pueda ejecutarse, como por ejemplo:

```
void main() {  
  print(22 / 7);  
}
```

si solo se escribe `22 / 7` en el editor de DartPad y se pulsa  **Run** se obtiene un error de compilación.

El operador `/` es la división real, si se quiere la **división entera**, hay que usar el operador `~/`

La expresión:

```
22 ~/ 7
```

se evalúa como:

```
3
```

Otro operador interesante es el operador **módulo** o **resto de la división** `%`

```
28 % 10
```

se evalúa como:

```
8
```

El orden de precedencia de los operadores es el normal: multiplicación y división tienen mayor precedencia que suma y resta. A igualdad de precedencia se aplican de izquierda a derecha. Si se quiere alterar este orden de precedencia se usan paréntesis.

Nota: Dart ignora los espacios en blanco en las expresiones, por lo que la expresión:

```
22 / 7
```

también puede escribirse como:

```
22/7
```

y se evalúa exactamente igual.

Funciones matemáticas

Para poder usar las funciones matemáticas en Dart hay que usar previamente la directiva:

```
import 'dart:math';
```

`math` es una biblioteca de Dart. Al usar la directiva `import` se le dice a Dart que se quieren usar las constantes o funciones de esa biblioteca.

Entre las funciones más importantes se encuentran:

- Raíz cuadrada: `sqrt`
- Seno: `sin`
- Coseno: `cos`
- Máximo: `max`
- Mínimo: `min`

Las funciones trigonométricas esperan argumentos en radianes. Si se quiere usar un argumento en grados sexagesimales, hay que realizar la conversión. Por ejemplo, para calcular el seno de 45° hay que escribir:

```
sin(45 * pi / 180)
```

La biblioteca `math` también incluye algunas constantes de utilidad, como por ejemplo:

```
pi // Tienen el valor 3.141592653589793
```

Variables

El siguiente fragmento de código:

```
int numero = 10;
```

Es una sentencia que declara una variable llamada `numero` de tipo `int` y la inicializa con el valor 10.

Existe también el tipo `double` que permite almacenar valores reales.

Importante: En Dart no existen tipos simples como en otros lenguajes. **Todos los tipos en Dart son clases**, incluidos `int` y `double` que son subclases de la clase `num`.

En Dart el operador `=` se usa para denotar asignaciones. No confundir con el operador `==` que es el operador de igualdad (se estudiará en próximas secciones).

Que esos tipos son clases se comprueba fácilmente con estos ejemplos:

```
10.isEven // Devuelve verdadero  
3.1416.round() // Devuelve 3
```

como puede verse, en ambos casos se han usado llamadas a métodos y propiedades de las clases `int` y `double`, directamente sobre valores de esas clases. Es así, porque `10` y `3.1416` son objetos.

Dart es un lenguaje de programación con **seguridad de tipos** (type-safe language). Esto significa que solo se puede asignar a una variable un valor de algún tipo compatible. Por ejemplo:

```
int miEntero = 10;
miEntero = 15; // OK
miEntero = pi; // ERROR, pi es una constante de tipo double
```

Otro ejemplo:

```
num miNumero = 10;
miNumero = 15; // OK
miNumero = pi; // OK, num es superclase también de double
miNumero = 'diez'; // ERROR, `diez` es una cadena de caracteres
```

Nombres de variable con sentido

Muy importante: Siempre se deben dar nombres con significado por sí mismos a las variables. Por ejemplo usar nombres de variables como:

```
numeroDePersonas
mediaPonderadaNotas
fechaSuscripcion
```

nunca usar nombres como:

```
np
media
f
```

En Dart el estándar es escribir los nombres de variable con el estilo `lowerCamelCase` que tiene las siguientes reglas:

- Cada nombre comienza siempre con letra minúscula.
- Si está formado por varias palabras, cada nueva palabra va junto a la anterior y en mayúscula.
- Las abreviaturas se tratan como si fueran palabras. Por ejemplo: `sourceUrl` o `urlDescription` en lugar de: `sourceURL` o `URLDescription`

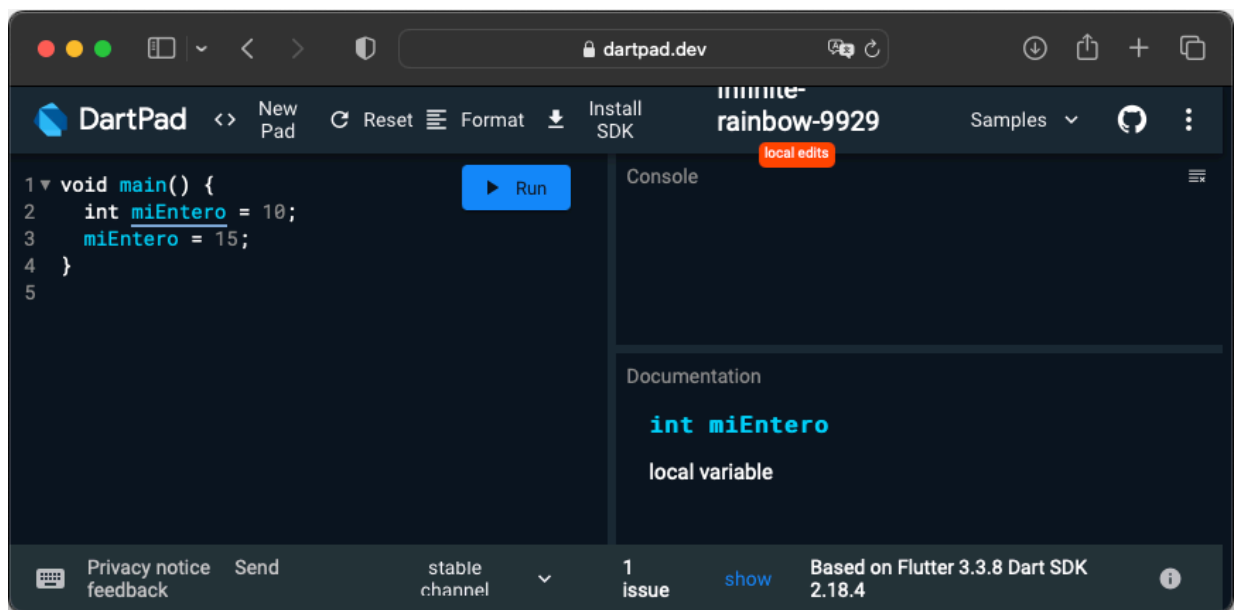
Inferencia de tipos

En Dart no es necesario escribir explícitamente el tipo de una variable en aquellos casos en los que se puede **inferir**. Por ejemplo:

```
var numero = 10;
```

La variable `numero` tiene tipo `int` porque el valor que se le asigna es de ese tipo.

En DartPad se puede comprobar el tipo de una variable simplemente haciendo clic sobre su nombre. En la zona de documentación se muestra el tipo inferido así como su ámbito.



Aunque no se haya definido explícitamente el tipo, el lenguaje sigue proporcionando seguridad de tipos:

```
var numero = 10;  
numero = 15; // OK ya que 15 es otro int  
numero = 3.14159; // ERROR
```

const y final

Las **constantes** son variables que contienen datos inmutables (su valor no cambia durante la ejecución del programa).

En Dart se usan dos palabras reservadas para definir constantes: `const` y `final`.

Por ejemplo:

```
const miConstante = 10;
```

define una constante llamada `miConstante` de tipo entero cuyo valor es 10 y no cambia.

```
const miConstante = 10;  
miConstante = 15; // ERROR. No se puede volver a asignar valor a una constante
```

Nota: En este ejemplo, se ha usado también **inferencia de tipos**, aunque también se puede declarar el tipo de una constante de forma explícita.

Existen constantes cuyo valor no cambia a lo largo de la ejecución del programa, pero cuyo valor concreto no se conoce **en tiempo de compilación**. Para definir estas constantes se usa `final`

```
final horasDesdeMedianoche = DateTime.now().hour;
```

Si se declarase usando `const` se obtendría el siguiente error de compilación:

```
Error: Method invocation is not a constant expression.
```

Al igual que las constantes definidas con `const`, las constantes definidas con `final` no pueden cambiar su valor después de haber sido inicializadas.

```
horasDesdeMedianoche = 0; //ERROR
```

Otro uso muy interesante de `final` es permitir separar la declaración de una variable de su inicialización:

```
final unNumero;

unNumero = 10; // OK porque unNumero está declarado pero aún no se ha inicializado
unNumero = 15; // ERROR porque ya tenía un valor y no se puede cambiar
```

Incrementar y decrementar

En Dart existen varios atajos para incrementar o decrementar el valor de una variable.

Las sentencias:

```
var counter = 0;
counter = counter + 1;
```

pueden abreviarse como:

```
counter += 1;
counter++;
```

También en el caso de las resta:

```
counter -= 1;
counter--;
```

Y también existen versiones para los operadores `*` y `/`

```
double unValor = 10;
unValor *= 3; // contiene el valor 30.0
unValor /= 2; // contiene el valor 15.0
```

Nota: `/=` produce un valor real, por tanto `unValor` se ha declarado como `double` ya que de otra forma la última sentencia produciría un error.

Mini ejercicios

1. Sabemos que el seno de 45° es igual a $\frac{1}{\sqrt{2}}$. Escribe un programa en Dart para comprobarlo.

Nota: Por ahora puedes imprimir los resultados por pantalla para ver el resultado de cada operación. Próximamente aprenderemos a usar condicionales y funciones.

2. El siguiente fragmento de código en Dart no funciona:

```
edad = 22;  
print(edad);  
edad = 30;  
print(edad);
```

Indica por qué no funciona y arréglalo. ¿Cuál de las siguiente palabras reservadas has usado `var`, `int`, `final` o `const`? ¿Por qué?

3. Dado el siguiente código:

```
const x = 46;  
const y = 10;
```

intenta adivinar mentalmente cuál es el valor de las siguientes constantes:

```
const answer1 = (x * 100) + y;  
const answer2 = (x * 100) + (y * 100);  
const answer3 = (x * 100) + (y / 10);
```

Programalo en Dart y comprueba si el resultado coincide con tu respuesta.

4. ➡ Una ecuación cuadrática tiene la forma:

$$ax^2 + bx + c = 0$$

Y la solución se se calcula como:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Escribe un programa en Dart donde los valores para `a`, `b` y `c` se definan como constantes y se calcule el resultado de la ecuación cuadrática usando la fórmula anterior. Almacena los resultados en dos variables llamadas `raiz1` y `raiz2`

¿De qué tipo has definido las constantes? ¿y las variables? ¿Por qué?

Dart 2: Tipos y operaciones

Contenidos

- Dart 2: Tipos y operaciones
 - Comprobación de tipos en tiempo de ejecución
 - Conversión de tipos
 - Operaciones con mezcla de tipos
 - Asegurar un tipo específico
 - Casting
 - Cadenas de caracteres
 - Concatenación
 - Interpolación
 - Cadenas con varias líneas
 - Tipos dinámicos
 - Mini ejercicios

Comprobación de tipos en tiempo de ejecución

El lenguaje Dart proporciona mecanismos para comprobar el tipo de una variable en código:

```
num unNumero = 3.14;
print(unNumero is double); // true
print(unNumero is int);    // false
```

Otra opción para comprobar el tipo de una variable es usar la propiedad `runtimeType` que tienen todas las clases en Dart y que devuelve el tipo **en tiempo de ejecución**:

```
print(unNumero.runtimeType); // double
```

A pesar de que `unNumero` se ha declarado de tipo `num`, en ese momento de la ejecución contiene un valor de tipo `double`. Su tipo en tiempo de ejecución es `double`.

Conversión de tipos

El siguiente fragmento de código en Dart no funciona:

```
var unEntero = 100;
var unDouble = 12.5;
unEntero = unDouble;
```

Se produce el error de compilación:

```
Error: A value of type 'double' can't be assigned to a variable of type 'int'.
```

Eso es debido a que Dart no realiza la conversión de tipos de forma implícita como hacen otros lenguajes de programación, ya que a menudo, es fuente de bugs difíciles de detectar.

Lo siguiente también es un error:

```
unDouble = unEntero;
```

Para realizar esas asignaciones se usan los métodos:

```
unEntero = unDouble.toInt();
```

y

```
unDouble = unEntero.toDouble();
```

Importante: De esta forma, la conversión de tipos se realiza bajo nuestro control y si los tipos son incompatibles, el error se produce en **tiempo de compilación** más visible y fácil de arreglar.

Nota: Cuando se usa `toInt()`, se produce una pérdida de precisión, ya que el tipo `int` no puede contener decimales y se pierden. Los decimales **se truncan** no se redondean.

Operaciones con mezcla de tipos

Se pueden escribir operaciones en las que los operandos tengan tipos mezclados. Por ejemplo:

```
const precioHora = 19.5;
const horasTrabajadas = 10;
const precioTotal = precioHora * horasTrabajadas; // double
```

El tipo que Dart infiere para `precioTotal` es `double` porque es la elección más segura. Si se elige `int` podría haber pérdida de precisión.

Si se necesita que el resultado sea de tipo `int`, se puede usar el método `toInt()`. Sin embargo el siguiente fragmento de código en Dart no funciona:

```
const precioTotal = (precioHora * horasTrabajadas).toInt(); // ERROR
```

Recordemos que no se puede usar `const` con constantes cuyo valor sea producto de una función o método y que por tanto, su valor no se conozca en tiempo de compilación. Para solucionarlo, se define la constante mediante `final`:

```
final precioTotal = (precioHora * horasTrabajadas).toInt(); // OK
```

Asegurar un tipo específico

Si se quiere una constante de tipo `double` cuyo valor inicial sea 3, no se puede escribir de la forma:

```
const quieroUnDouble = 3; // int
```

porque, como sabemos, Dart infiere que el tipo es `int` dado que 3 es un **literal entero**.

Para conseguir el efecto buscado se pueden emplear las siguientes técnicas:

```
final esUnDouble = 3.toDouble();
const double esUnDouble = 3;
const esUnDouble = 3.0;
```

Cualquiera de las tres produce el resultado deseado. En los tres casos, Dart infiere que el tipo de `unDouble` es justamente `double`.

Casting

En ocasiones se tiene una variable de una superclase, pero se necesita funcionalidad que está disponible solo en las subclases. En estos casos se puede **forzar el tipo** con la palabra reservada `as`. Veamos un ejemplo:

```
num unNumero = 3;
```

La propiedad `isEven` está disponible solo para los enteros, por lo que lo siguiente no funcionará:

```
print(unNumero.isEven);
```

dado que `unNumero` es de tipo `num`. Se produce el error en tiempo de compilación:

```
Error: The getter 'isEven' isn't defined for the class 'num'.
```

Si se está seguro de que en una variable definida con el tipo de una superclase, hay un valor de alguna de las subclases, se puede forzar a Dart a tratar la variable como si se hubiera definido con el tipo de esa subclase concreta.

En este caso se podría acceder a la propiedad `isEven` de esta forma:

```
final unEntero = unNumero as int;  
print(unEntero.isEven); // Perfectamente válido
```

o escribiéndolo directamente:

```
print((unNumero as int).isEven);
```

La palabra reservada `as` fuerza al compilador a reconocer `unNumero` como variable de tipo `int`

Hay que tener cuidado al hacer **casting de tipos**. Por ejemplo, lo siguiente no funciona:

```
final unString = unEntero as String;
```

Se produce el error en tiempo de ejecución:

```
type 'int' is not a subtype of type 'String' in type cast
```

ya que la clase `String` no es una subclase de la clase `int`.

Cuidado: El mensaje de error dice todo lo contrario lo que puede ser un poco confuso.

Cadenas de caracteres

En Dart una cadena de caracteres puede escribirse tanto entre comillas simples `' '` como entre comillas dobles `" "`.

Nota: En Dart no se recomienda usar ninguna en particular, pero la guía de estilo de Flutter recomienda usar comillas simples, por eso son las que usaremos aquí.

Así para definir una variable de tipo cadena de caracteres mediante inferencia de tipos, podemos escribir:

```
var cadena = 'Hola Mundo!';
```

Si en un literal de tipo cadena de caracteres definido mediante comillas simples, hay que escribir el símbolo de las comillas simples, hace falta precederlo del carácter `\`. Por ejemplo:

```
var cadena = 'My cat\'s food';
```

Las cadenas en Dart son **inmutables**. Si se escribe:

```
var cadena = 'Hola Mundo!';  
cadena = 'Adiós Mundo!';
```

La primera cadena se descarta por completo, se crea otra cadena de caracteres **completamente nueva** con el contenido `'Adiós Mundo!'` y se asigna a la variable `cadena`.

Importante: Esto es importante si se van a realizar frecuentes asignaciones de cadenas, ya que **afecta al rendimiento** de la aplicación.

Concatenación

Se pueden construir cadenas de caracteres más complejas concatenando otras cadenas de caracteres con el operador `+`

```
var mensaje = 'Hola';  
mensaje = mensaje + ' Mundo!';
```

También usando la sintaxis:

```
mensaje += ' Mundo!';
```

Nota: Como es lógico, aquí `cadena` se ha declarado como variable y no como constante, ya que de otra forma no se podría haber concatenado.

Si se tienen que hacer muchas concatenaciones para construir una cadena, por ejemplo, construir una cadena con todos los datos de un empleado, es mejor usar un `StringBuffer` que es **mucho más eficiente**:

```
final mensaje = StringBuffer();
mensaje.write('Hola');
mensaje.write(' mi nombre es');
mensaje.write(' Francisco');
```

Esto crea una ubicación **mutable** en la memoria donde se puede añadir nuevas subcadenas a la cadena **sin tener que crear una nueva cadena para cada cambio**. Cuando se ha terminado de usar un `StringBuffer`, se puede usar `toString()` para convertir el `StringBuffer` al tipo `String`.

```
String otroMensaje = mensaje.toString();
```

Importante: lo siguiente no funciona

```
mensaje.toString();
String otroMensaje = mensaje;
```

ya que `toString()` devuelve un nuevo `String` con el contenido del `StringBuffer`, no modifica al `StringBuffer`.

Interpolación

Otra forma de construir cadenas de caracteres es mediante **interpolación**. Consiste en sustituir partes de una cadena por otros valores.

```
int miEdad = 55;
final cadenaEdad = 'Mi edad es: $miEdad';
print(cadenaEdad); // Mi edad es: 55
```

Se usa el carácter `$` delante del nombre de la variable cuyo valor se quiere sustituir.

```
const unTercio = 1 / 3;
const frase = 'Un tercio es: $unTercio.';
print(frase); // Un tercio es: 0.3333333333333333.
```

Puede ser que no se necesiten tantos número decimales. En ese caso se puede usar `toStringAsFixed`

```
final frase = 'Un tercio es: ${unTercio.toStringAsFixed(3)}.';
print(frase); // Un tercio es: 0.333.
```

Importante: Dado que se está llamando al método `toStringAsFixed()` sobre la constante `unTercio` y lo que se quiere sustituir es el valor después de aplicar el método, se usan llaves `{ }` para indicar a Dart qué es lo que se debe sustituir. De no hacerlo, el `$` solo afectaría a la constante `unTercio` y se produciría la siguiente salida en la consola:

```
Un tercio es: 0.3333333333333333.toStringAsFixed(3).
```

Importante: Como en ejemplos anteriores, en este segundo caso la constante `frase` no puede declararse como `const` sino que debe ser declarada como `final`.

Cadenas con varias líneas

Se pueden usar cadenas con varias líneas. Para ello se usan tres comillas simples `'''` al principio y al final de la cadena multilínea.

Por ejemplo, el siguiente fragmento de código en Dart:

```
const stringGrande = '''
Se puede tener una cadena
que contenga multiples
líneas
haciendo esto.''';
print(stringGrande);
```

imprime el siguiente resultado en la consola:

```
Se puede tener una cadena
que contenga multiples
líneas
haciendo esto.
```

Importante: No deben sangrarse las líneas interiores de la cadena porque aparecen espacios no deseados.

Otra forma de conseguir cadenas de caracteres con varias líneas es usando `\n` en el interior de la cadena. Por ejemplo, el siguiente fragmento de código en Dart:

```
const dosLineas = 'En esta cadena\nhay dos líneas';
print(dosLineas);
```

imprime en la consola de depuración:

```
En esta cadena
hay dos líneas
```

Tipos dinámicos

Existe la posibilidad de definir variables con tipos dinámicos. Son variables que pueden contener cualquier tipo en tiempo de ejecución.

Son útiles solo en circunstancias muy concretas y generan muchos problemas, por lo que no se estudiarán en este repaso de Dart.

Mini ejercicios

1. Define una constante en Dart llamada `edad` y asígnale el valor 42 sin especificar el tipo. Define otra constante llamada `estatura` y asígnale el valor 1.80 sin especificar el tipo.

Comprueba, directamente en DartPad, los tipos que Dart ha inferido para ambas constantes. ¿Cómo lo has hecho?

2. Crea dos constantes llamadas `edad1` y `edad2` y asígnales los valores 10 y 5 respectivamente. Calcula su media aritmética y asigna el resultado a otra constante llamada `mediaEdad`.

¿Qué tipo tiene la constante `mediaEdad`? ¿A qué crees que se debe?

3. Intenta adivinar mentalmente cuál será la salida impresa en la consola, del siguiente fragmento de código en Dart:

```
num miEntero = 10;
print(miEntero is int);
print(miEntero is double);
print(miEntero is num);
```

a continuación prográmalo en DartPad y compara el resultado obtenido con tu respuesta.

4. ¿Cuál es el error en el siguiente fragmento de código en Dart?

```
const nombre = 'Francisco';
nombre += ' Conde Rodriguez';
```

¿Cómo lo arreglarías?

5. 🕒 El siguiente fragmento de programa en Dart no funciona:

```
const precioHora = 19.5;
const horasTrabajadas = 10;
const precioTotal = (precioHora * horasTrabajadas).toInt();
```

Explica ¿por qué?. ¿Cómo lo resuelves? Justifica tu respuesta.

Dart 3: Control de flujo

- Dart 3: Control de flujo
 - Valores booleanos
 - Operadores lógicos
 - Operadores booleanos
 - Igualdad de cadenas de caracteres
 - La orden if
 - Ámbito de las variables
 - Operador condicional ternario
 - La orden switch
 - Evitar el uso excesivo de switch
 - Tipos enumerados
 - Bucles
 - Bucle while
 - Bucle do-while
 - Diferencia entre while y do-while
 - Rompiendo un bucle
 - Un ejemplo con random
 - Bucle for
 - La palabra reservada continue
 - Iterando sobre los valores de un tipo enumerado
 - Mini ejercicios

Valores booleanos

El tipo en Dart para almacenar valores booleanos es `bool` y solo puede tomar dos valores `true` y `false`. Por ejemplo:

```
const si = true;
const no = false;
```

Operadores lógicos

El operador de igualdad es `==`

```
const sonIgualesUnoYDos = (1 == 2); // false
```

La siguiente comparación también es posible, ya que tanto `int` como `double` son subclases de `num`:

```
int unEntero = 3;
double unDouble = 3.0;
bool iguales = (unEntero == unDouble); // true
```

El operador de desigualdad es `!=`

```
const esUnoDistintoDeDos = (1 != 2); // true
```

La expresión anterior, también podría escribirse como:

```
const esUnoDistintoDeDos = !(1 == 2); // true
```

en este caso, se usa el operador uno-ario `!`, que recibe el nombre de operador de **negación lógica**. Cambia el valor `true` a `false` y el valor `false` a `true`.

Los demás operadores lógicos habituales `<`, `>`, `<=` y `>=` también se usan en Dart.

Importante: Como ocurre en la mayoría de lenguajes de programación, en Dart, los operadores lógicos tienen menor prioridad que los operadores aritméticos.

Operadores booleanos

Los resultados de operaciones lógicas pueden combinarse entre sí, por medio de los operadores booleanos **AND** (que se escribe `&&`) y **OR** (que se escribe `||`) y cuyo comportamiento es el habitual.

Por ejemplo la expresión:

```
true && false || true
```

da como resultado `true`.

Las **reglas de precedencia** para los operadores booleanos y lógicos son:

!
>= > <= <
== !=
&&

Los operadores que se sitúan en posiciones superiores en esa tabla, tienen mayor prioridad que los que se sitúan en posiciones inferiores. Por ejemplo, el operador **AND** `&&` tiene mayor prioridad que el operador **OR** `||`.

Como es habitual, esas reglas de precedencia pueden alterarse mediante el uso de paréntesis.

Igualdad de cadenas de caracteres

En Dart se puede comparar si dos cadenas de caracteres son iguales o distintas usando los operadores `==` y `!=` respectivamente.

```
const animal = 'perro';  
const animalIgualGato = (animal == 'gato'); // false
```

Importante: Los operadores `<`, `<=`, `>` y `>=` no se pueden aplicar al tipo `String`.

Para realizar esa comparación se usa el método de la clase `String` llamado `compareTo` que devuelve -1 si la cadena a la que se aplica va antes, en orden alfabético, que el argumento; 0 si las cadenas son iguales y 1 si va después en orden alfabético.

Por ejemplo el siguiente fragmento de código:

```
const animal = 'perro';  
final orden = animal.compareTo('gato'); // 1
```

asigna a la constante `orden` el valor `1`, ya que la cadena de caracteres `perro` va detrás en orden alfabético, que la cadena `gato`.

La orden if

Una forma de alterar el flujo de ejecución de un programa en Dart es mediante la orden `if` que permite ejecutar selectivamente fragmentos de código en función de si se cumple o no una condición. La forma más simple de `if` es:

```
if (2 > 1) {  
    print('Sí, 2 es mayor que 1');  
}
```

La parte escrita entre paréntesis es la condición, que es **siempre una expresión lógica** que puede ser resultado de combinar otras expresiones lógicas mediante operadores booleanos.

En Dart no pueden escribirse condiciones como la siguiente:

```
int valor = 1;  
if (valor) {  
    // cuerpo del if  
}
```

que sí son posibles en otros lenguajes de programación.

En cualquier caso, no es una gran pérdida, ya que esta clase de condiciones oscurece el código y hace que sea mucho menos legible, por lo que no deben utilizarse.

En Dart pueden usarse cláusulas `else` y `else if` de la forma habitual:

```
const semaforo = 'amarillo';  
var accion = '';  
if (semaforo == 'rojo') {  
    accion = 'Parar';  
} else if (semaforo == 'amarillo') {  
    accion = 'Frenar';  
} else if (semaforo == 'verde') {  
    accion = 'Avanzar';  
} else {  
    accion = 'Color no válido!';  
}  
print(accion);
```

Ámbito de las variables

En el siguiente fragmento de programa se definen variables en tres ámbitos distintos:

```
const global = 'Hola, mundo';

void main() {
  const local = 'Hola, main';

  if (2 > 1) {
    const deltroDelIf = 'Hola, hay alguien?';

    print(global);
    print(local);
    print(deltroDelIf);
  }

  print(global);
  print(local);
  print(deltroDelIf); // Error
}
```

- La variable `global` está definida fuera de la función `main()` y de cualquier par de llaves `{}`. Es una variable global o top-level variable. Está accesible desde cualquier punto **dentro del archivo**. Es peligroso definir este tipo de variables.
- La variable `local` está definida dentro del cuerpo de la función `main()` por lo que está disponible desde cualquier punto de esa función. Es una variable local.
- La variable `deltroDelIf` está definida dentro del cuerpo de la orden `if` dentro de la función `main()` por lo que solo puede accederse desde el cuerpo del `if`. Cuando se intenta imprimir el contenido de `deltroDelIf` desde el cuerpo de la función `main()` pero fuera del cuerpo del `if` se da un error de compilación, porque en ese ámbito, la función `deltroDelIf` no está definida.

Nota: Si se quiere averiguar el ámbito de una variable en DartPad, se puede hacer clic sobre su nombre. En la zona de documentación de DartPad, aparece junto al tipo inferido, su ámbito (top-level o local)

Operador condicional ternario

En Dart también está definido el operador condicional ternario:

```
(condicion) ? valorSiVerdadero : valorSiFalso;
```

Pero su uso no es recomendable, ya que el código es mucho menos legible y es igual de eficiente.

La orden switch

Cuando se tienen **múltiples condiciones**, la orden `if` puede ser difícil de escribir y entender. En estos casos es mejor usar la orden `switch`.

```
switch (variable) {  
  case value1:  
    // código  
    break;  
  case value2:  
    // código  
    break;  
  
  ...  
  
  default:  
    // código  
}
```

- La variable de control del `switch` puede ser de tipo `int` o de tipo `String`. En función del valor de esa variable se dirigirá el flujo a alguno de los casos escritos a continuación.
- Cada caso se marca con la palabra reservada `case` seguida del valor y dos puntos `:`. Los valores de los casos se comparan con el de la variable mediante el operador `==`.
- Al final del cuerpo de un caso se usa la palabra reservada `break` para indicar a Dart que el bloque de código de ese caso ha terminado.
- Al final del `switch` se puede poner opcionalmente la palabra reservada `default` para indicar el caso especial en el que ninguno de los otros casos coincide con el valor de la variable. En ese caso se ejecutaría el código que se haya escrito a continuación de `default`.

Si se da la situación en que dos casos comparten el mismo código, es decir, en los dos casos se debe ejecutar exactamente el mismo conjunto de instrucciones, no hace falta repetirlas, se puede escribir así:

```
case value1:  
case value2:  
  // código compartido por ambos casos  
  break;
```

Como ya se ha comentado, se puede usar `switch` con variables de tipo `String`. Por ejemplo, el siguiente fragmento de código:

```
const weather = 'nublado';
switch (weather) {
  case 'soleado':
    print('Coge una gorra.');
```

```
    break;
  case 'nublado':
  case 'lluvioso':
    print('Coge un paraguas.');
```

```
    break;
  default:
    print("No estoy familiarizado con este tiempo.");
}
```

imprime en la consola `Coge un paraguas.`

Nota: En Dart, las órdenes `switch` no admiten rangos como por ejemplo `numero > 5`. Solo se permite la verificación de igualdad mediante el operador `==`. Si en un programa se necesitan condiciones que involucren rangos, entonces se debe usar la orden `if`.

Evitar el uso excesivo de switch

La orden `switch` es muy útil, pero a menudo se abusa de ella. Cuando se usa con clases, en muchas ocasiones es mejor sustituir el `switch` por **patrones de diseño de software** basados en **polimorfismo** como el patrón: **estrategia**.

Si en un programa se tienen muchos `switch` con muchos casos, a menudo es síntoma de que el código es de baja calidad. Muchos de esos `switch` deberían refactorizarse para sustituirlos por el patrón estrategia.

Nota: En la sesión 3 se explican algunos patrones de diseño de software y cómo se programan en Dart.

Tipos enumerados

Dart también permite definir tipos enumerados, por ejemplo:

```
enum Meteorologia {
  soleado,
  lluvioso,
  nublado
}
```

`soleado`, `lluvioso` y `nublado` son las **constantes del tipo enumerado**. Cualquier variable que se defina de tipo `Meteorologia` solo podrá tomar alguno de esos tres valores.

El criterio para nombrar las constantes del tipo enumerado es el mismo que para nombrar variables. Se usa la notación `lowerCamelCase`, por lo que las constantes que tengan una sola palabra van en minúsculas.

A las constantes de un tipo enumerado se accede a través del nombre del tipo.

```
Meteorologia.soleado
```

En Dart los tipos enumerados **no son enteros**, sin embargo, se puede obtener un entero que indica su posición dentro de la lista de constantes del tipo enumerado. Para ello se usa la propiedad `index`. Por ejemplo:

```
enum Meteorologia {
  soleado,
  lluvioso,
  nublado
}

void main() {
  const tiempo = Meteorologia.lluvioso;
  print(tiempo); // Meteorologia.lluvioso
  print(tiempo.index); // 1, ya que el índice de la primera constante del enum es 0
}
```

Nota: Si se quiere probar en DartPad algún ejemplo que incluya tipos enumerados, la declaración del tipo debe ir fuera de la función `main` o de lo contrario se tendrá un error de compilación.

Los tipos enumerados funcionan especialmente bien con las órdenes `switch`.

```
const tiempo = Meteorologia.nublado;
switch (tiempo) {
  case Meteorologia.soleado:
    print('Coge una gorra');
    break;
  case Meteorologia.nublado:
  case Meteorologia.lluvioso:
    print('Coge un paraguas');
}
```

En este caso no es necesario usar la palabra reservada `default` ya que se han definido casos para todas las constantes del tipo.

Importante: No es obligatorio que todas las constantes de un tipo enumerado aparezcan como casos en un `switch`. Si para alguna constante no se ha definido caso, y la variable tiene justamente como valor a esa constante, no se ejecuta el código de ningún caso del `switch`.

Bucles

Bucle while

Un bucle `while` repite un bloque de código mientras el valor de una condición sea `true`. Su sintaxis es:

```
while (condicion) {  
    // bloque de código que se repite  
}
```

Un bucle `while` repite el código escrito en el cuerpo del bucle, un número indeterminado de veces, ya que depende de `condicion` que es una expresión booleana. Mientras la condición sea `true` se itera dentro del ciclo.

Un ejemplo sencillo es:

```
var contador = 1;  
while (contador <= 10 ) {  
    print(contador);  
    contador += 4;  
}
```

que define un bucle `while` que itera tres veces y que imprime en la consola:

```
1  
5  
9
```

Si no se actualiza la condición en el interior del cuerpo del bucle, o no se escribe correctamente la condición, se puede tener un bucle que no termina nunca o un **bucle infinito**.

Bucle do-while

De forma similar, un bucle `do-while` repite un bloque de código mientras el valor de una condición sea `true`. Su sintaxis es:

```
do {  
    // código que se repite  
} while (condicion);
```

Los bucles `do-while` también repiten el código escrito en el cuerpo del bucle un número indeterminado de veces. Depende de `condicion` que es una expresión booleana. Mientras la condición sea `true` se itera dentro del bucle.

Un ejemplo sencillo es:

```
var contador = 1;
do {
    print(contador);
    contador += 4;
} while (contador <= 10); // OJO con el ; al final de esta instrucción
```

que tiene el mismo efecto que el bucle `while` de la sección anterior.

Importante: Hay que escribir `;` al final de la condición en un bucle `do-while` ya que si no, se produce un error de compilación.

Diferencia entre while y do-while

En el bucle `while` la condición se comprueba **al comienzo** del bucle, por lo que podría ser que fuera `false` al principio y el bucle no se ejecute ni una sola vez.

En el bucle `do-while` la condición se comprueba **al final** del bucle, por lo que el cuerpo del bucle se ejecuta al menos una vez.

Rompiendo un bucle

En el cuerpo de un bucle `while` o `do-while` puede usarse la palabra reservada `break` que interrumpe inmediatamente la ejecución del ciclo y continúa con el código escrito justo después del bucle. Por ejemplo, el siguiente fragmento de código:

```
var suma = 1;
while (true) {
    suma += 4;
    print(suma);
    if (suma > 10) {
        break;
    }
}
```

imprime en la consola:

```
5
9
13
```

En este ejemplo, la condición del bucle es `true` por lo que el bucle nunca pararía de iterar y se tendría un bucle infinito. Sin embargo, la instrucción `break` en el caso de que `suma` sea mayor que 10 interrumpe la ejecución del bucle y se continúa con la siguiente instrucción.

Un ejemplo con random

Nota: Para usar números aleatorios en Dart es necesaria la biblioteca matemática a la que se accede con la orden:

```
import 'dart:math';
```

Imaginemos que se necesita simular las tiradas de un dado con seis caras, hasta obtener un seis.

El primer paso es construir un generador de números aleatorios:

```
final aleatorio = Random();
```

La variable `aleatorio` contiene un generador de números aleatorios que puede usarse para obtener números aleatorios mediante el método `nextInt(int max)` que genera un número entero aleatorio no negativo uniformemente distribuido en el rango de 0, incluido, a `max`, no incluido.

```
aleatorio.nextInt(6)
```

Es decir, la instrucción anterior genera un entero aleatorio en el intervalo $[0,5]$ que también se puede escribir como $[0,6)$

Ahora vamos a escribir un ejemplo que usa un ciclo `while` para simular tiradas de dados hasta que salga un valor 6.

```
final aleatorio = Random();  
while (aleatorio.nextInt(6) + 1 != 6) {  
    print('No ha salido un 6');  
}  
print('Finalmente has logrado un 6');
```

Una posible salida de ese programa es:

```
No ha salido un 6  
No ha salido un 6  
No ha salido un 6  
Finalmente has logrado un 6
```

Nota: Se usa un generador de números aleatorios por lo que el número concreto de veces que se obtiene el mensaje `No ha salido un 6` variará en cada ejecución.

Bucle for

En Dart también se pueden usar bucles al estilo C. Para ello se usa la palabra reservada `for`. Su sintaxis es:

```
for (var i = 0; i < 5; i++) {  
    print(i);  
}
```

A continuación de la palabra reservada `for` se escriben entre paréntesis y separados por `;` tres grupos de caracteres:

- `var i = 0` define la **variable de control del bucle**, en este caso `i`. Se debe inicializar con algún valor. En el ejemplo se ha inicializado con el valor 0.

Importante: La palabra reservada `var` es **necesaria**, ya que si no se pone y la variable `i` no se ha declarado fuera del bucle, se estaría asignando valor a una variable no declarada, lo que es un error.

También se puede usar `int` o `double` en vez de `var`.

- `i < 5` es la **condición**. Se comprueba antes de cada iteración del bucle. Si se evalúa con el valor `true` entonces se ejecuta el cuerpo del bucle (escrito entre llaves `{ }` y se sigue iterando). Si se evalúa como `false`, se termina la ejecución del bucle y se continúa por la siguiente instrucción.
- `i++` es la **acción** que se ejecuta al final de cada iteración. Lo normal es que actualice el valor de la variable de control del bucle. Debe escribirse con cuidado, ya que de lo contrario se podría tener un bucle infinito que no termina nunca.

También se puede inicializar la variable de control del bucle **antes** del bucle.

```
int i;  
for (i = 0; i < 5; i++) {  
    print(i);  
}
```

En ambos casos el programa realiza la misma tarea, se escribe en la consola los números enteros del 0 al 4 incluidos.

La diferencia, es que en el primer caso, la variable de control del ciclo está definida en el ámbito del cuerpo del ciclo, y no puede usarse fuera de éste, mientras que en el segundo caso, la variable de control del ciclo `i` sigue existiendo una vez que se termina la ejecución del bucle.

La palabra reservada continue

Se puede usar la palabra reservada `continue` para saltar alguna de las iteraciones de un bucle `for`.

Por ejemplo, el siguiente fragmento de código:

```
for (var i = 0; i < 5; i++) {  
  if (i == 2) {  
    continue;  
  }  
  print(i);  
}
```

imprime en la consola de depuración la siguiente salida:

```
0  
1  
3  
4
```

El valor 2 no se imprime, porque se saltó la ejecución de esa iteración mediante la palabra reservada `continue`.

Iterando sobre los valores de un tipo enumerado

En Dart se puede iterar fácilmente sobre todos los valores de un tipo enumerado. Para ello se usa un método especial de las colecciones llamado `forEach()`. Este método se estudia más adelante en el bloque dedicado a las colecciones.

Mini ejercicios

1. Crea una contante llamada `edad` y asígnale un valor cualquiera. A continuación crea otra constante llamada `esAdolescente` y usa la lógica booleana para calcular si la edad contenida en `edad` está dentro del rango de 13 a 19 años.
2. Crea ahora otra constante llamada `edadDeAmigo` y asígnale otro valor. A continuación crea otra constante llamada `ambosAdolescentes` y usa la lógica booleana para calcular si ambos están en el rango de edad entre 13 y 19 años.
3. Crea una contante llamada `edad` y asígnale un valor. A continuación escribe un `if` para que escriba `Adolescente` si la edad está en el rango de 13 a 19 años y `No adolescente` en caso contrario.
4. Intenta averiguar, sin programarlo, ¿cuál es el error del siguiente fragmento de código? ¿cómo lo podrías arreglar?

```
const nombre = 'Fran';  
if (nombre == 'Fran') {  
  const apellido = 'Rodríguez';  
} else if (nombre == 'Pedro') {  
  const apellido = 'Muñoz';  
}  
final nombreCompleto = nombre + ' ' + apellido;
```

5. Crea un tipo enumerado llamado `EstadoReproductor` con los casos `parado` , `en pausa` y `reproduciendo` .
6. Crea una constante del tipo enumerado `EstadoReproductor` y asígnale un valor. A continuación usa un `switch` para escribir un mensaje basado en el valor de la variable.
7. ¿Cuántas veces se ejecuta el cuerpo del siguiente bucle? ¿Con qué valores?

```
var valor = 1;
while(valor < 10) {
    valor *= 2;
    print(valor);
}
```

8. Escribe un bucle `do-while` que tenga exactamente el mismo efecto que el bucle anterior.
9. Intenta averiguar, sin programarlo, ¿cuál es el error en este bucle? ¿Cómo lo arreglas?

```
var sum = 1;
do {
    print(sum);
    sum += 4;
} while(sum < 10)
```

10. ➡ Intenta averiguar, sin programarlo, ¿cuál es el error en este otro bucle?

```
for (i = 0; i<10; i++) {
    print(i);
}
```

11. ¿De cuántas formas se puede resolver el error del ejercicio anterior? ¿Qué diferencias hay entre ellas? ¿Cuál es preferible?
12. Escribe el bucle más sencillo posible para imprimir en la consola la siguiente salida:

```
0.0
0.1
0.2
0.3
0.4
0.5
```

13. Escribe un bucle `for` que imprima el número y su raíz cuadrada con 4 decimales de los números comprendidos entre 1 y 10 incluidos. Utiliza la interpolación de cadenas de caracteres vista en secciones anteriores.

Nota: La función matemática que permite realizar la raíz cuadrada es `sqrt`

14. 📌 Escribe una constante llamada `numero` y asígnale un valor entero positivo. A continuación escribe el código necesario para imprimir por pantalla la siguiente potencia de 2 de `numero`.

Ejemplo: si el número introducido es 23, debe imprimir en la consola 32, ya que es el siguiente entero mayor que 23 que es potencia de 2, en concreto $32 = 2^5$

Usando solo funciones matemáticas, condicionales y bucles, escribe el código que se ejecute con el menor número de operaciones.

15. 📌 Escribe un fragmento de código que calcule el n-ésimo elemento de la serie de Fibonacci.

Nota: La serie de Fibonacci comienza con 1, el segundo elemento es 1 de nuevo, y los elementos posteriores se calculan sumando los dos previos. Por ejemplo, el tercer elemento es $1 + 1 = 2$

Dart 4: Funciones

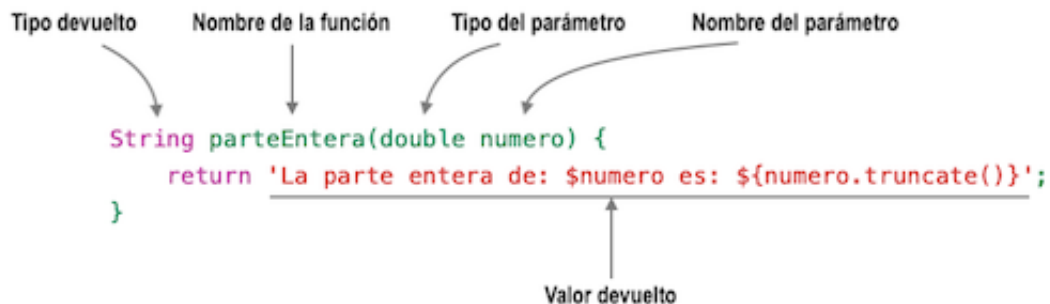
- Dart 4: Funciones
 - Funciones
 - Anatomía de una función
 - Parámetros
 - Parámetros opcionales
 - Parámetros con nombre
 - Funciones anónimas
 - Asignar funciones a variables
 - Pasar funciones a otras funciones como argumentos
 - Devolver una función como resultado de otra función
 - Clausuras y alcance
 - Notación flecha
 - Mini ejercicios

Funciones

Como cualquier lenguaje de programación, en Dart es posible declarar y usar funciones.

Anatomía de una función

En Dart una función tiene las siguientes partes:



- **Tipo devuelto:** Si la función devuelve algún valor, su tipo es la primera palabra de la declaración de la función. Si la función realiza un proceso pero no devuelve ningún valor, entonces se usa la palabra reservada `void`.
- **Nombre de la función:** Lo escoge el programador, debería seguir la notación `lowerCamelCase`.
- **Parámetros:** Son los datos de entrada para la función. Se escriben entre paréntesis justo a continuación del nombre de la función. Si hay más de un parámetro, se separan mediante comas `,`. Para cada parámetro, primero se escribe su tipo y a continuación su nombre. Los nombres de parámetros siguen las mismas reglas que los nombres de variables y también se usa la notación `lowerCamelCase`.
- **Valor que se devuelve:** La última instrucción que se ejecuta en una función es `return` y va seguida de algún valor cuyo tipo debe coincidir con el tipo devuelto de la función. Si el tipo devuelto es `void` no hay que escribir la orden `return`.

El tipo devuelto, nombre de la función y parámetros se conocen como **signatura** de la función. En este ejemplo:

```
String parteEntera(double numero)
```

es la signatura de la función.

El código escrito entre las llaves se conoce como el **cuerpo** de la función. En este ejemplo, el cuerpo de la función es:

```
{  
    return 'La parte entera de: $numero es: ${numero.truncate()}';  
}
```

En el contexto de un programa, la función anterior se escribiría así:

```
void main() {  
    const valor = 8.1;  
    print(parteEntera(valor));  
}  
  
String parteEntera(double numero) {  
    return 'La parte entera de: $numero es: ${numero.truncate()}';  
}
```

En ese fragmento de código, en realidad hay dos funciones. La función `main` es también una función. Cuando se inicia la ejecución de un programa en Dart, lo que se ejecuta es justo esa función.

La función `parteEntera` está fuera de la función `main`. Dart soporta **top-level functions**, que son funciones que no están declaradas dentro de ninguna otra función o clase.

En Dart también se pueden definir funciones dentro del cuerpo de otra función. En este caso se denominan **nested functions**.

También se pueden definir funciones dentro de clases en cuyo caso se llaman **métodos**. Hablaremos de los métodos en la siguiente sección.

Para llamar a una función se escribe su nombre y sus argumentos entre paréntesis.

Importante: Aunque a veces se confunden ambos términos, un **parámetro** es el tipo y el nombre de una variable de entrada que se define cuando se **declara** la función, mientras que un **argumento** es el valor concreto que se pasa cuando se **llama** a la función.

Así por ejemplo, en el fragmento de código anterior, `numero` es un parámetro, mientras que `8.1` es un argumento.

Parámetros

Cuando se necesita que una función tenga más de un parámetro, se escriben separados por comas ,

```
void holaPersonaYMascota(String persona, String mascota) {  
    print('Hola, $persona, y tu peludo amigo, $mascota!');  
}
```

En esta función ambos parámetros son de tipo `String`, si no se tiene cuidado podrían confundirse.

Un ejemplo de uso de esta función es:

```
holaPersonaYMascota('Francisco', 'Rambo');
```

que produce la siguiente salida en la consola:

```
Hola, Francisco, y tu peludo amigo, Rambo!
```

Parámetros opcionales

En ocasiones no es necesario pasar todos los argumentos a una función. Con la sintaxis descrita en la sección anterior hay que escribirlos todos siempre en la llamada. Esto es muy poco flexible.

Imagina una función que recibe como argumentos un nombre, un apellido y un título, y devuelve una cadena de caracteres que los combina. Podría tener este aspecto:

```
String nombreCompleto(String nombre, String apellidos, String titulo) {  
    return '$titulo: $nombre $apellidos';  
}
```

Si se ejecuta con los siguientes argumentos:

```
print(nombreCompleto('Francisco', 'Conde Rodriguez', 'Profesor doctor'));
```

se muestra en la consola:

```
Profesor doctor: Francisco Conde Rodriguez
```

La función funciona correctamente, pero es muy poco flexible. ¿Qué ocurre si en un contexto concreto, no todas las personas tienen un título, o no es relevante? La solución de Dart es usar **parámetros opcionales**.

Para marcar un parámetro como opcional, se escribe entre corchetes `[ ]`.

```
String nombreCompleto(String nombre, String apellidos, [String? titulo]) {  
    if (titulo != null) {  
        return '$titulo: $nombre $apellidos';  
    } else {  
        return '$nombre $apellidos';  
    }  
}
```

`[String? titulo]` es un parámetro opcional. Los corchetes indican que es opcional (podemos escribirlo o no al llamar a la función) y el símbolo `?` indica que **puede tener un valor nulo**. Por eso es necesario el `if` en el cuerpo de la función.

Con esta versión de la función, si escribimos la llamada:

```
print(nombreCompleto('Francisco', 'Conde Rodriguez'));
```

obtendremos como resultado:

```
Francisco Conde Rodriguez
```

Otra posibilidad que ofrece Dart al usar parámetros opcionales es usar **valores por defecto** distintos de `null`. El valor por defecto para un parámetro se escribe a continuación de su nombre y separado por el símbolo igual `=`. Por ejemplo:

```
bool dentroDelIntervalo(int valor, [int min = 0, int max = 10]) {  
    return (min <= valor) && (valor <= max);  
}
```

Nota: En este caso, no es necesario poner el símbolo `?` ya que el parámetro no es `null`, siempre va a tener un valor de tipo `int`, bien porque se escriba al llamar a la función, bien porque no se escriba y se tome el valor por defecto.

Si no se escriben los argumentos, se toman los valores por defecto.

```
dentroDelIntervalo(5) // true  
dentroDelIntervalo(15) // false
```

También se pueden usar valores distintos a los valores por defecto.

```
dentroDelIntervalo(9, 7, 11) // true
```

Sin embargo, esta otra forma de especificar los argumentos también es válida y puede ser confusa. Por ejemplo:

```
dentroDelIntervalo(9, 7) // true
```

en este caso, se ha especificado un segundo argumento, el valor `7`, que siempre se refiere al parámetro `min`. Con esta notación, no es posible escribir solo el parámetro `max` y que `min` tome el valor por defecto. Un mecanismo más flexible son los parámetros con nombre.

Parámetros con nombre

El mecanismo de Dart para que los parámetros de una función tengan un significado más claro son los **parámetros con nombre** que se escriben entre llaves `{}`.

Por ejemplo, la siguiente versión de la función `dentroDelIntervalo` usa parámetros con nombre:

```
bool dentroDelIntervalo(int valor, {int min = 0, int max = 10}) {  
    return (min <= valor) && (valor <= max);  
}
```

Dart, por defecto, considera que un parámetro con nombre es opcional (podemos o no escribirlo en la llamada), aunque esto puede cambiarse.

Nota: Se puede hacer que parte de los parámetros de la función sean parámetros con nombre, y parte parámetros posicionales normales.

La forma en que se llama a una función que tenga parámetros con nombre es:

```
dentroDelIntervalo(9, min: 7, max: 11) // true
```

A la hora de llamar a la función, los argumentos con nombre deben ir precedidos del nombre del parámetro y dos puntos :

Esta forma es mucho más flexible. Las siguientes llamadas a la función también son correctas:

```
dentroDelIntervalo(9, min: 7, max: 11) // true
dentroDelIntervalo(9, max: 11, min: 7) // true

dentroDelIntervalo(5)           // true
dentroDelIntervalo(15)          // false

dentroDelIntervalo(5, min: 7)   // false
dentroDelIntervalo(15, max: 20) // true
```

Por defecto **Dart considera que los parámetros con nombre son opcionales**. Si se quiere que el parámetro `valor` también sea un parámetro con nombre, por defecto Dart también lo considera opcional, con lo que la siguiente llamada sería válida:

```
dentroDelIntervalo(); // No tiene sentido
```

pero esta llamada no tiene mucho sentido. ¿Cuál es el valor para el que se quiere saber si está dentro o no del intervalo?

En estos casos se usa la palabra reservada `required`.

```
bool dentroDelIntervalo({required int valor, int min = 0, int max = 10}) {
    return (min <= valor) && (valor <= max);
}
```

que hace que el parámetro `valor` deba escribirse siempre.

Por ejemplo:

```
dentroDelIntervalo(valor: 5) // true
```

Importante: El uso de parámetros con nombre hace que el código sea más legible y es una herramienta fundamental para escribir código limpio cuando una función tiene múltiples parámetros del mismo tipo.

Funciones anónimas

Todas las funciones vistas hasta ahora son **funciones con nombre**, sin embargo, en Dart también se pueden declarar **funciones anónimas**.

Una función anónima no tiene ni tipo devuelto ni nombre de función. Por ejemplo:

```
String parteEntera(double numero) {  
    return 'La parte entera de: $numero es: ${numero.truncate()}';  
}
```

En una función anónima, el **tipo devuelto se infiere** del valor que se devuelve en el cuerpo de la función, en este ejemplo `String`.

Eliminar el nombre y el tipo de valor devuelto permite tratar la función anónima resultante como un valor.

Importante: Las funciones anónimas no pueden declararse y usarse tal cual, sino que tienen usos muy concretos que se ven en los apartados siguientes.

Importante: Todos estos usos de las funciones anónimas pueden parecer confusos y poco importantes, pero son **fundamentales** para poder escribir programas para móviles en Flutter, por lo que hay que aprenderlos bien.

Asignar funciones a variables

Una razón por la que se necesitan funciones anónimas es que no se puede asignar una función con nombre a una variable.

Para poder asignar una función a una variable, ésta debe ser de tipo `Function`. Por ejemplo:

```
Function multiplicar = (int a, int b) {  
    return a * b;  
};
```

En este ejemplo, la variable de tipo `Function` llamada `multiplicar` contiene una función anónima que recibe dos argumentos de tipo `int` y devuelve su producto.

Para usar la función contenida en la variable, solo hay que escribir el nombre de la variable y la lista de argumentos. Por ejemplo:

```
final resultado = multiplicar(10, 5); // asigna a resultado el valor 50
```

El tipo que Dart infiere para la variable `resultado` es `int` que es el tipo devuelto por la función anónima.

Si la función anónima no tiene parámetros se ejecuta así:

```
Function saludo = () {  
    print('Hola mundo!');  
};  
  
saludo();
```

Pasar funciones a otras funciones como argumentos

También es posible pasar a una función con nombre una función anónima como uno de sus argumentos. La sintaxis es la siguiente:

```
void funcionConNombre(Function funcionAnonima) {  
    // cuerpo de la función con nombre donde se usa la función anónima.  
}
```

Se podría construir esta otra función con nombre, que recibe una función anónima sin argumentos como uno de sus parámetros:

```
void unGranSaludo(Function unSaludo) {  
    unSaludo();  
}
```

y llamarla con la función anónima guardada en la variable `saludo` de la sección anterior:

```
unGranSaludo(saludo);
```

Pero también se podría declarar esta otra función anónima y asociarla a la variable `buenosDias` :

```
Function buenosDias = () {  
    print('Buenos días!');  
};
```

y usarla con la función `unGranSaludo` de la forma:

```
unGranSaludo(buenosDias);
```

Importante: Todas las posibles funciones anónimas que se usen como argumentos tienen que tener obligatoriamente **la misma lista de argumentos y el mismo tipo devuelto**.

Devolver una función como resultado de otra función

De la misma forma que se pueden pasar funciones anónimas como argumentos a una función con nombre, una función con nombre también puede devolver una función anónima como resultado. El tipo devuelto es `Function` .

Un ejemplo de función con nombre que devuelve una función anónima como resultado es:

```
Function creaMultiplicador(num multiplicador) {  
    return (num valor) {  
        return valor * multiplicador;  
    };  
}
```

Este mecanismo puede parecer un poco confuso. Vamos a verlo en acción para entender su funcionamiento. Para ello vamos a escribir la siguiente declaración:

```
final multiplicaPor3 = creaMultiplicador(3);
```

Es importante darse cuenta de que:

- `multiplicaPor3` es una variable que contiene una función anónima. La que devolvió la función con nombre `creaMultiplicador` en la declaración anterior.
- El argumento `3` se pasa a la función con nombre `creaMultiplicador` luego corresponde con el parámetro `multiplicador` de ésta función.
- Al llamar a la función `creaMultiplicador` solo se ejecuta el primero de los `return`, el que construye y devuelve la función anónima resultado de `creaMultiplicador`. El segundo `return` se ejecutará cuando se llame a la función anónima.

Ahora vamos a ver cómo usar la función anónima almacenada en la variable `multiplicaPor3`:

```
print(multiplicaPor3(4)); // 12  
print(multiplicaPor3(8.0)); // 24.0
```

Es importante darse cuenta de que:

- En la variable de tipo `Function` llamada `multiplicaPor3` está la función anónima que devolvió la función con nombre `creaMultiplicador`. Recordemos que es:

```
(num valor) {  
    return valor * multiplicador;  
};
```

por tanto cuando se quiera ejecutar hay que pasarle un argumento de tipo `num`, en los ejemplos `4` y `8.0`.

- Ese argumento se hace corresponder con el parámetro llamado `valor`
- Cuando se ejecuta, se toma el argumento que se pasa en el parámetro `valor` y se multiplica por `multiplicador` cuyo valor es siempre `3`, ya que ese valor se asignó al construir la función anónima en la declaración de la variable `multiplicaPor3`.

Nota: Las funciones que toman a otras funciones como argumentos, o que devuelven funciones como resultado se denominan **higher order functions**, y son **obligatorias** para usar programación dinámica en Dart y para construir interfaces avanzadas en Flutter.

Clausuras y alcance

Todas las funciones anónimas en Dart actúan como **clausuras** porque su código se cierra alrededor del alcance que la rodea. En Dart, un alcance es lo que se escribe entre llaves `{ }`

En el ejemplo de la función `creaMultiplicador`, la función anónima definida como tipo devuelto, se cierra sobre el alcance de la propia función `creaMultiplicador`. Eso significa que tiene acceso a todas las variables y funciones definidas en ese alcance, en concreto, tiene acceso al parámetro `multiplicador`.

Veamos otro ejemplo:

```
var contador = 0;

final incrementarContador = () {
  contador += 1;
};

print(contador); // 0

incrementarContador();
incrementarContador();
incrementarContador();

print(contador); // 3
```

Como puede verse, la función anónima almacenada en `incrementarContador`, a pesar de no tener parámetros, tiene acceso a la variable `contador` y puede manipularla. Es porque su ámbito (al ser una función anónima) se cierra sobre el alcance que la rodea, que incluye a la variable `contador`

Por eso se dice que las funciones anónimas en Dart actúan como clausuras.

Importante: Una cosa es declarar la función y otra distinta es usarla. Al declarar la función no se incrementa `contador`. Esto ocurre si se ejecuta. Por eso, la primera orden `print` muestra en la consola el valor `0`

Notación flecha

Dart tiene una sintaxis especial para funciones cuyo cuerpo es solo una línea. Es lo que se conoce como **notación flecha**.

La siguiente función con nombre `suma` recibe dos argumentos de tipo `int` y devuelve su suma:

```
int suma(int a, int b) {
  return a + b;
}
```

Dado que el cuerpo de esa función es solo una línea, puede reescribirse usando notación flecha de la siguiente forma:

```
int suma(int a, int b) => a + b;
```

Nota: Esta última forma solo implica un cambio en la indentación de la función lo que permite una forma más compacta de escribir la función. La función escrita con notación flecha no es más eficiente o mejor que si está escrita en el otro formato.

Importante: La notación flecha se puede aplicar tanto a funciones con nombre como a funciones anónimas.

Importante: No es posible usar `return` cuando se usa notación flecha, ya que va implícito.

Mini ejercicios

1. Escribe una función llamada `eresGenial` con un parámetro de tipo `String` llamado `nombre` que devuelva un `String` que use el valor de `nombre` y diga algo como: Eres genial, Francisco!
2. Modifica la función anterior para que además de recibir un nombre, reciba un parámetro de tipo `int` llamado `numeroPersonas` y diga algo como: Eres genial, Francisco!, 10 personas lo piensan.
3. Haz que ambos parámetros sean parámetro con nombre. el parámetro `nombre` debe escribirse siempre y el parámetro `numeroPersonas` debe ser opcional con valor por defecto 30.
4. 🗳️ ¿Cuál de estas dos funciones es mejor? ¿Explica por qué?

```
void hola() {  
    print('Hola!');  
}  
  
String hola() {  
    return 'Hola!';  
}
```

5. Escribe una función llamada `areaCirculo` que calcule el área de un círculo cuyo radio se pasa como argumento.

Nota: el área de un círculo de radio r se calcula como πr^2

6. Escribe una función con la siguiente signature:

```
bool esPrimo(int numero)
```

Que devuelva `true` si el número pasado es primo y `false` en caso contrario.

Nota: Un número es primo si solo es divisible por sí mismo y por 1.

Consejo: Puede ser muy útil tener una función auxiliar llamada `esDivisible` que acepte dos parámetros `numero` y `divisor` y que devuelva `true` si `numero` es divisible por `divisor` y `false` en caso contrario.

7. ➡ Modifica la función `eresGenial` del primer ejercicio para que sea anónima. Asígnala a una variable llamada `genial`.
8. La función llamada `devuelveFuncion` devuelve como salida una función anónima:

```
Function devuelveFuncion() {  
  return () {  
    print('Hola Mundo!');  
  };  
}
```

Estúdiala y explica por qué de las dos llaves del final `}`, una lleva punto y coma `;` y la otra no.

9. Explica por qué en la primera declaración no se usa punto y coma `;` después de la última llave y en la segunda sí:

```
int suma(int a, int b) {  
  return a + b;  
}
```

```
var suma = (int a, int b) {  
  return a + b;  
};
```

10. ➡ Escribe una función llamada `repetirTarea` con la siguiente signatura:

```
int repetirTarea(int veces, int valor, Function tarea)
```

que repita la tarea que se pasa en el parámetro `tarea`, sobre el valor pasado en `valor`, tantas veces como indica `veces`.

Usa esa función para multiplicar un número por sí mismo varias veces.

¿Cómo cambiarías la signatura de la función con nombre `repetirTarea` para al llamarla, el significado de sus argumentos sea mucho más claro?

Dart 5: Clases

- Dart 5: Clases
 - Clases en Dart
 - Definir clases
 - Construir objetos de una clase
 - Asignar valores a las propiedades
 - Imprimir un objeto
 - Notación en cascada
 - Constructores
 - Constructor por defecto
 - Constructores personalizados
 - Constructores con nombre
 - Reenvío de constructores
 - Parámetros opcionales y con nombre
 - Propiedades privadas
 - Métodos privados
 - Constructores privados
 - Propiedades inmutables
 - Constructores inmutables
 - Constructores factoría (Factory constructors)
 - Objetos en Dart
 - getters
 - Propiedades calculadas
 - Setters
 - Miembros de clase
 - Mini ejercicios

Clases en Dart

Dart es un lenguaje orientado a objetos. Todos los valores en Dart son objetos que se han declarado a partir de alguna clase. Incluso los tipos como `int` o `double` que son primitivos en otros lenguajes, son clases en Dart. Por esta razón, saber utilizar las clases es fundamental para poder sacar todo el partido al lenguaje Dart.

Definir clases

Una clase en Dart, en su forma más simple, se define así:

```
class Usuario {  
  int id = 0;  
  String nombre = '';  
}
```

`id` y `nombre` son las **propiedades** de esa clase.

En esta clase solo se han definido dos propiedades y ningún método, ni siquiera constructores, sin embargo es plenamente utilizable.

Construir objetos de una clase

A los objetos contruidos a partir de una clase también se los denomina **instancias**. Por esa razón al proceso de construir objetos a partir de clases, también se le llama **instanciar** una clase.

Para instanciar una clase propia se sigue el mismo proceso que se ha visto para construir objetos a partir de las clases del sistema como `int` o `String`. Por ejemplo:

```
final usuario = Usuario();
```

Importante: Como puede verse, en las clases propias, también se da inferencia de tipos.

Nota: La palabra reservada `new` era obligatoria antes de la versión 2.0 de Dart, sin embargo, en las versiones actuales no es necesaria, así que mejor no ponerla para no complicar el código.

```
final usuario = new Usuario(); // Válido, pero no necesario. Mejor no poner
```

Asignar valores a las propiedades

Para acceder a las propiedades de un objeto se usa la notación `.` de la forma habitual. Por ejemplo:

```
class Usuario {
  int id = 0;
  String nombre = '';
}

void main() {
  final usuario = Usuario();
  usuario.nombre = 'Francisco';
  usuario.id = 42;
}
```

Nota: Tanto la función `main` como la clase `Usuario` están definidas en el mismo fichero. No es lo deseable, en próximas sesiones de prácticas veremos como modularizar el código.

Nota: En Dart el orden en que se escriban la función `main` y la clase no es relevante. En este ejemplo la función aparece debajo, pero también podría haberse escrito encima.

Imprimir un objeto

También se pueden imprimir en la consola los objetos de las clases propias como por ejemplo la clase `Usuario`, aunque el resultado es poco útil.

```
print(usuario);
```

escribe en la consola:

```
Instance of 'Usuario'
```

Importante: Todas las clases en Dart se derivan de la superclase `Object` en la que se define el método `toString`. La función `print` usa este método para obtener una cadena de caracteres a partir del objeto e imprimirla.

La implementación por defecto del método `toString` en la clase `Object` es devolver que el objeto es instancia de la clase correspondiente.

Si se quiere obtener información más relevante cuando se imprime un objeto de la clase `Usuario` hay que redeclarar el método `toString`. Por ejemplo:

```
@override
String toString() {
  return 'Usuario(id: $id, nombre: $nombre)';
}
```

Las palabras reservadas en Dart que comienzan con `@` se llaman **anotaciones**. No cambian la forma en que se ejecuta el código, sino que dan al compilador la información que necesita para realizar su trabajo correctamente. En este caso, se le dice al compilador que se intenta redeclarar un método de la superclase. Si al escribir la signature del método lo hacemos incorrectamente, el compilador nos avisará de que no hay ningún método en la superclase con esa signature y podremos corregir el error fácilmente.

Si se añade la redeclaración anterior al final de la clase `Usuario` y se escribe la siguiente orden al final de la función `main`:

```
print(usuario);
```

se obtiene la salida:

```
Usuario(id: 42, nombre: Francisco)
```

Nota: Es costumbre entre programadores colocar el método `toString` al final de la clase, donde no molesta mucho y es sencillo de localizar.

Notación en cascada

En ejemplos anteriores se ha definido un objeto de la clase `Usuario` y se han modificado sus propiedades de la forma:

```
final usuario = Usuario();
usuario.nombre = 'Francisco';
usuario.id = 42;
```

Dart define el operador en cascada `..` que permite simplificar un poco el código anterior:

```
final usuario = Usuario()
  ..nombre = 'Francisco'
  ..id = 42;
```

Importante: En la notación en cascada, el punto y coma `;` solo debe aparecer en la última línea.

Constructores

Un constructor es un método que construye instancias de una clase, es decir, son funciones de la clase que generan nuevos objetos. Los constructores tienen el mismo nombre de la clase y su tipo devuelto va implícito, ya que es la propia clase.

Consejo: En Dart existen diferentes tipos de constructores y su sintaxis es diferente a la de otros lenguajes de programación por lo que deberías estudiar con detenimiento esta sección.

Constructor por defecto

La clase `Usuario` usada anteriormente no tiene un constructor explícito. En estos casos, Dart proporciona un **constructor por defecto** que no tiene parámetros y que simplemente devuelve una instancia de la clase. Por ejemplo:

```
class Usuario {  
  int id = 0;  
  String nombre = '';  
}
```

es equivalente a:

```
class Usuario {  
  int id = 0;  
  String nombre = '';  
  
  Usuario();  
}
```

Consejo: En Dart es opcional incluir en una clase el constructor por defecto, por ello, lo mejor es no ponerlo ya que simplifica el código.

Constructores personalizados

Se pueden pasar parámetros al constructor para modificar cómo la clase construye al objeto. La sintaxis es la misma vista para los parámetros de las funciones.

Al igual que en el constructor por defecto, el nombre de este tipo de constructor personalizado, debe ser el mismo que el nombre de la clase.

Nota: Tanto a los constructores por defecto, como a los constructores personalizados, se les denomina **constructores generativos** ya que generan directamente un objeto del mismo tipo.

Importante: Si se escribe un constructor personalizado, Dart no proporciona el constructor por defecto, **por lo que no puede usarse**.

Hay dos formas de escribir un constructor personalizado, la forma larga y la corta.

Esta es la forma larga:

```
Usuario(int id, String nombre) {  
  this.id = id;  
  this.nombre = nombre;  
}
```

`this` es una palabra reservada que hace referencia al propio objeto que se construye y permite usar el mismo nombre para las propiedades de la clase y para los parámetros del constructor, ya que al usarla se elimina la ambigüedad.

También existe una forma corta, que es completamente equivalente a la anterior:

```
Usuario(this.id, this.nombre);
```

y mucho más sencilla de escribir.

En ambos casos, para crear una instancia de `Usuario` se usa la misma sintaxis. Por ejemplo:

```
final usuario = Usuario(42, 'Francisco');
```

Constructores con nombre

Dart también tiene un segundo tipo de constructor generativo que son los **constructores con nombre**. Sirven para construir objetos con necesidades especiales, como por ejemplo, construir un tipo especial de usuario, el usuario anónimo. En este ejemplo, podría definirse un nuevo constructor llamado `anonimo` de la siguiente forma:

```
class Usuario {  
  int id = 0;  
  String nombre = '';  
  
  Usuario(this.id, this.nombre);  
  Usuario.anonimo() {  
    id = 0;  
    nombre = 'Anónimo';  
  }  
  
  // Aquí el resto de métodos  
}
```

El identificador o nombre de este constructor con nombre es `.anonimo`.

Importante: Los constructores con nombre pueden tener parámetros. En este ejemplo no eran necesarios. La sintaxis para los parámetros es la que ya conocemos.

Nota: En este ejemplo, no hay parámetros, por lo que tampoco hay confusión a la hora de inicializar las propiedades. Por eso no se ha usado la palabra reservada `this`

Para crear una instancia de `Usuario` usando el constructor con nombre se escribe:

```
final usuarioAnonimo = Usuario.anonimo();  
print(usuarioAnonimo);
```

Al ejecutar el fragmento de código anterior, se obtiene la salida en la consola de depuración:

```
Usuario(id: 0, nombre: Anónimo)
```

Reenvío de constructores

Como puede verse, el código en el cuerpo del constructor personalizado y el constructor con nombre es bastante parecido. Eso está violando el principio de diseño de software conocido como **Don't Repeat Yourself** o **DRY**.

Dart proporciona herramientas para resolver esto. El reenvío de constructores consiste en llamar al constructor personalizado (el constructor más genérico) desde el constructor con nombre.

Para ello el constructor con nombre se reescribe así:

```
Usuario.anonimo() : this(0, 'Anónimo');
```

El constructor con nombre no tiene cuerpo, sino que llama al constructor personalizado.

Importante: Cuando se usa reenvío de constructores, la palabra reservada `this` sustituye al nombre de la clase, en este ejemplo: `Usuario`.

La forma de llamar al constructor con nombre es exactamente la misma vista anteriormente:

```
final usuarioAnonimo = Usuario.anonimo();
```

Ya no es necesaria la **inicialización de las propiedades** porque todos los constructores garantizan que todas ellas van a estar inicializadas, por lo que podemos sustituir:

```
int id = 0;  
String nombre = '';
```

por:

```
int id;  
String nombre;
```

Con todo ello la clase `Usuario` se puede reescribir así;

```
class Usuario {  
  int id;  
  String nombre;  
  
  Usuario(this.id, this.nombre);  
  Usuario.anonimo() : this(0, 'Anónimo');  
  
  // Aquí el resto de métodos  
}
```

Parámetros opcionales y con nombre

Todo lo que se ha estudiado a propósito de los parámetros de las funciones, es válido también para los parámetros de los constructores.

Se puede escribir un constructor con parámetros opcionales usando los corchetes `[ ]`:

```
MiClase([this.miPropiedad = 0]);
```

Con parámetros con nombre y opcionales usando las llaves `{ }`:

```
MiClase({this.miPropiedad = 0}); // Se debe igualar al valor inicial que interese
```

Y con parámetros con nombre, pero requeridos (no opcionales) usando las llaves `{ }` junto a la palabra reservada `required`:

```
MiClase({required this.miPropiedad});
```

En el ejemplo visto anteriormente de la clase `Usuario` se puede redefinir el constructor personalizado en forma corta:

```
Usuario(this.id, this.nombre);
```

de forma que los parámetros sean opcionales y tengan nombre:

```
Usuario({this.id = 0, this.nombre = 'Anónimo'});
```

En este caso, el constructor personalizado realiza la misma tarea que el constructor con nombre `.anonimo` por lo que puede reescribirse de la forma:

```
Usuario.anonimo() : this();
```

La clase completa quedaría así:

```
class Usuario {
    int id;
    String nombre;

    Usuario({this.id = 0, this.nombre = 'Anónimo'});
    Usuario.anonimo() : this();

    // Aquí el resto de métodos
}
```

La llamada al constructor generativo personalizado cambia. Ahora es obligatorio escribir los nombres de los parámetros:

```
final usuario = Usuario(id: 42, nombre: 'Francisco');
```

sin embargo, la llamada al constructor generativo con nombre permanece tal cual porque no tenía parámetros:

```
final usuarioAnonimo = Usuario.anonimo();
```

Propiedades privadas

Tal y como se ha definido la clase `Usuario` hasta este momento, presenta algunos problemas. Veamos el siguiente fragmento de código:

```
final vicki = Usuario(id: 24, nombre: 'Vicki');  
vicki.nombre = 'Nefarious Hacker';  
print(vicki); // Usuario(id: 24, nombre: Nefarious Hacker)
```

Como podemos ver, se ha declarado el objeto `vicki` como `final` ¿cómo es entonces posible que se puedan cambiar los valores de sus propiedades?

Porque lo que es constante es la instancia, es decir, no es posible asignar una nueva instancia a la variable `vicki`. Si, a continuación del fragmento de código anterior, escribimos:

```
vicki = Usuario(id: 42, nombre: 'Francisco');
```

obtendremos el siguiente error de compilación:

```
Error: Can't assign to the final variable 'vicki'.
```

Para impedir que se pueda modificar el valor de las propiedades de una clase una vez instanciada, Dart permite definir las propiedades de una clase como **privadas**. Para ello solo hay que escribir un guión bajo `_` delante del nombre de la propiedad.

El siguiente fragmento de código:

```
int _id;  
String _nombre;
```

declara las propiedades `_id` y `_nombre` como privadas.

Sin embargo, Dart no permite que los parámetros con nombre comiencen por `_`, por lo que el constructor personalizado debe reescribirse, ya que si se escribe así:

```
Usuario({this._id = 0, this._nombre = 'Anónimo'});
```

se obtiene el siguiente error de compilación:

```
Error: An optional named parameter can't start with '_'.
```

Por tanto, hay que reescribirlo con la siguiente sintaxis:

```
Usuario({int id = 0, String nombre = 'Anónimo'}):  
  _id = id,  
  _nombre = nombre;
```

A la lista separada por comas que va detrás de los dos puntos `:` se la llama **lista de inicialización**.

Importante: La lista de inicialización es lo primero que se ejecuta en este constructor. El cuerpo del constructor, si lo tiene, se ejecuta siempre después. Por ejemplo, si se define así el constructor:

```
Usuario({int id = 0, String nombre = 'Anónimo'}):  
  _id = id,  
  _nombre = nombre {  
    print('El nombre del usuario es $_nombre');  
  }
```

y se llama con la siguiente orden:

```
final vicki = Usuario(id: 24, nombre: 'Vicki');
```

la salida que se obtiene en la consola es:

```
El nombre del usuario es Vicki
```

Si la definición de la clase `Usuario` y la función `main` están en el mismo archivo, todavía es posible escribir:

```
final vicki = Usuario(id: 24, nombre: 'Vicki');  
vicki._nombre = 'Nefarious Hacker';
```

sin que se produzcan errores de compilación. ¿Cómo es posible si se han definido las propiedades de la clase como privadas?

Importante: Las propiedades privadas en Dart son **privadas para la biblioteca, no para la clase**. Si la función `main` y la clase `Usuario` están en el mismo archivo, todavía se tiene acceso a las propiedades desde la función `main`. Por esta razón la clase `Usuario` debería escribirse en su propio archivo.

En DartPad no es trivial trabajar con varios archivos en el mismo proyecto, por lo que en esta sesión todavía seguiremos escribiendo todo el código junto en un archivo. En próximas sesiones usaremos el IDE **Android Studio** y podremos separar cada clase en su propio archivo de forma sencilla.

Métodos privados

De la misma forma que se pueden definir propiedades privadas para una clase escribiendo un guión bajo `_` delante del nombre de la propiedad, también se pueden declarar métodos privados de la misma forma.

Por ejemplo, si se añade este método a la clase `Usuario`:

```
String _usuarioFormateado() => 'Usuario(id: $_id, nombre: $_nombre)';
```

se declara un método que solo es accesible desde otro método de la clase `Usuario` a la que pertenece, pero que no puede invocarse desde fuera de la clase.

Por ejemplo, el método `toString` podría reescribirse para que use el método anterior de la siguiente forma:

```
@override
String toString() {
    return _usuarioFormateado();
}
```

Nota: Los métodos privados se usan para realizar procesos que afectan a la lógica interna de la clase, sin exponerlos al exterior de la misma.

Constructores privados

Un tipo especial de métodos privados son los constructores privados. Son constructores que solo se pueden usar desde dentro de la clase en la que se definen.

Por ejemplo, si se tiene una clase llamada `UnaClase` se puede hacer que su constructor por defecto sea privado de la siguiente forma:

```
UnaClase._();
```

Nota: Puede parecer inútil que Dart proporcione la capacidad de tener constructores privados. Si los constructores de una clase son privados no se pueden generar instancias de esa clase, ¿qué interés puede tener una clase de la que no se pueden generar instancias?

La respuesta es que sin los constructores privados ciertas estructuras modernas de programación no serían posibles. Por ejemplo, no se podrían escribir clases que sigan el **patrón de diseño de software** llamado **Singleton**.

Aprenderás más sobre patrones de diseño de software y Singleton más adelante.

Propiedades inmutables

Como sabemos, Dart proporciona dos formas de indicar que una variable es constante (inmutable, su valor no puede cambiar una vez asignado), que son `const` y `final`. Sin embargo en las clases, **las propiedades solo pueden marcarse como `final`** ya que su valor no se conoce en tiempo de compilación.

Por ejemplo, se pueden reescribir las propiedades de la clase `Usuario` de la siguiente forma:

```
final int _id;  
final String _nombre;
```

Añadir la palabra `final` tiene como resultado que a `_nombre` e `_id` solo se les puede dar un valor una vez, es decir, cuando se llama al constructor. Una vez creado el objeto, esas propiedades serán inmutables.

Constructores inmutables

Si los objetos de una clase en particular no pueden cambiar, porque **todas las propiedades** de la clase son `final`, se puede añadir la palabra reservada `const` delante del nombre del constructor para asegurarse de que todas las instancias de la clase sean constantes en tiempo de compilación.

```
class Usuario {  
  final int _id;  
  final String _nombre;  
  
  const Usuario({int id = 0, String nombre = 'Anónimo'}):  
    _id = id,  
    _nombre = nombre;  
  const Usuario.anonimo() : this();  
  
  // Aquí el resto de métodos  
}
```

De esta forma se pueden declarar las instancias de la clase `User` como constantes en tiempo de compilación. Por ejemplo:

```
const usuario = Usuario(id: 42, nombre: 'Francisco');  
const usuarioAnonimo = Usuario.anonimo();
```

las variables `const` además de ser inmutables, son **instancias canónicas**. Eso significa que no importa cuantas instancias se hayan construido, si los argumentos usados para crearlas son los mismos, Dart solo crea una única instancia en memoria. Así por ejemplo, se puede llamar al constructor `Usuario.anonimo()` 100 veces en una aplicación sin que ello suponga un deterioro del rendimiento por tener que gestionar 100 objetos diferentes.

Importante: Usar objetos y constructores constantes tanto como sea posible, produce una gran mejora en el rendimiento de la aplicación.

Constructores factoría (Factory constructors)

Todos los constructores que se han visto hasta ahora han sido constructores generativos. Dart también proporciona otro tipo de constructor llamado **constructor factoría**.

Un constructor factoría proporciona mayor flexibilidad a la hora de crear objetos. Es básicamente un método especial que comienza con la palabra reservada `factory` y devuelve un objeto del tipo de la clase.

El caso más simple se implementa llamando en el cuerpo del constructor factoría a un constructor generativo. Por ejemplo:

```
factory Usuario.francisco() {  
    return Usuario(id: 42, nombre: 'Francisco');  
}
```

Aquí, el constructor factoría usa el constructor generativo para crear y devolver una nueva instancia de la clase `Usuario`,

Importante: En este ejemplo simple, se podría haber logrado el mismo efecto con un constructor con nombre. La ventaja es que en un constructor generativo, la inicialización de las propiedades se realiza **antes** de ejecutar cualquier otro código dentro del constructor, mientras que el constructor factoría no.

Un ejemplo más elaborado de constructor factoría para la clase `Usuario` puede ser:

```
factory Usuario.desdeString({String idString = '0', String nombre = ''}) {  
    int idNumero = int.parse(idString);  
    return Usuario(id: idNumero, nombre: nombre);  
}
```

Este constructor, permite realizar una conversión de tipos antes de construir el objeto. En este ejemplo, el `id` no se escribe directamente como un número entero sino como una cadena de caracteres.

Nota: `parse` es un método de clase, de la clase `int`, que recibe una cadena de caracteres y devuelve el entero que representa esa cadena. Si en la cadena que se pasa como argumento no representa a un número entero, entonces se lanza una excepción.

Por ejemplo, el siguiente fragmento de código:

```
final usuarioString = Usuario.desdeString(idString: '22', nombre: 'Pedro');  
print(usuarioString);
```

imprime en la consola de depuración:

```
Usuario(id: 22, nombre: Pedro)
```

Importante: En el cuerpo de un constructor factoría se puede realizar **cualquier proceso**, como validar argumentos, comprobación de errores o incluso modificación de los argumentos, **antes** de devolver el objeto, **sin exponer a quien lo usa** los entresijos del proceso de instanciación.

Objetos en Dart

En Dart, los objetos se asignan a las variables como **referencias**, esto significa que si se asigna un objeto a dos variables, las dos contienen referencias al mismo objeto en memoria.

Por ejemplo, si se tiene la clase:

```
class MiClase {  
    var MiPropiedad = 1;  
}
```

y se instancia de la siguiente forma:

```
final unObjeto = MiClase();  
final otroObjeto = myObject;
```

tanto `unObjeto` como `otroObjeto` contienen una referencia al mismo objeto en memoria. Eso se demuestra fácilmente con el siguiente fragmento de código:

```
print(unObjeto.MiPropiedad);    // 1  
otroObjeto.MiPropiedad = 2;  
print(unObjeto.MiPropiedad);    // 2
```

Nota: Si se necesita hacer una copia real de la clase, no solo una copia de su referencia en memoria, sino un objeto completamente nuevo con una **copia profunda** de todos los datos que contiene, entonces hay que implementar un método en la clase que construya un objeto completamente nuevo a partir de otro.

Una forma de implementar el método para hacer la copia es usar un constructor factoría. Por ejemplo, si se tiene la clase `Usuario` declarada de la forma:

```
class Usuario {  
    int id;  
    String nombre;  
  
    Usuario({this.id = 0, this.nombre = 'Anónimo'});  
    Usuario.anonimo() : this();  
  
    @override  
    String toString() => 'Usuario(id: $id, nombre: $nombre)';  
}
```

se le puede añadir el siguiente constructor factoría:

```
factory Usuario.copiaDe({required Usuario usuario}) {  
    return Usuario(id: usuario.id, nombre: usuario.nombre);  
}
```

que permite hacer **copias profundas** de los objetos de la clase `Usuario`. Veamos un ejemplo:

```
final unUsuario = Usuario(id: 45, nombre: 'Francisco');  
final otroUsuario = Usuario.copiaDe(usuario: unUsuario);  
  
print(unUsuario);  
print(otroUsuario);  
  
otroUsuario.id = 0;  
otroUsuario.nombre = 'Anónimo';  
  
print(unUsuario);  
print(otroUsuario);
```

El resultado en la consola es:

```
Usuario(id: 45, nombre: Francisco)  
Usuario(id: 45, nombre: Francisco)  
Usuario(id: 45, nombre: Francisco)  
Usuario(id: 0, nombre: Anónimo)
```

ya que en este caso no se tienen referencias sino objetos diferentes en memoria.

getters

Si en una clase se tienen propiedades privadas, no se puede acceder a ellas desde otro módulo de la aplicación. El mecanismo de Dart para permitir el acceso desde fuera de la clase a las propiedades privadas se denomina **getters**.

Un getter en Dart es un método especial que usa la palabra reservada `get` delante del nombre de una propiedad y devuelve un valor.

Por ejemplo, si se tiene la clase `Usuario` definida de esta forma:

```
class Usuario {  
    final int _id;  
    String _nombre;  
  
    Usuario({int id = 0, String nombre = 'Anónimo'}):  
        _id = id,  
        _nombre = nombre;  
    Usuario.anonimo() : this();  
  
    @override  
    String toString() => 'Usuario(id: $id, nombre: $nombre)';  
}
```

sus propiedades son privadas. La única forma de acceder a ellas desde fuera de la clase es añadiendo getters, como por ejemplo:

```
int get id => _id;  
String get nombre { return _nombre; }
```

Como puede verse, se puede usar tanto notación flecha, que es la que se ha usado en el getter de la propiedad `_id`, como notación estándar, que es la que se ha usado en el getter de `_nombre`.

En ambos casos el efecto es el mismo, se devuelve el valor de la propiedad, por lo que puede usarse cualquiera de las dos notaciones.

La forma en que se usan los getters es mediante notación punto. Por ejemplo:

```
final usuario = Usuario(id: 45, nombre: 'Francisco');  
print(usuario.id); // 45  
print(usuario.nombre); // Francisco
```

En este caso no hay error de compilación al acceder a `id` o `nombre` porque la clase `Usuario` tiene getters para ambas.

Propiedades calculadas

Se pueden escribir getters que no accedan a un valor de propiedad, sino que el valor que devuelven se calcule en el momento de la llamada. A este tipo de getters se les denomina **propiedades calculadas**.

Por ejemplo, imaginemos que se reservan los primeros 1000 enteros para identificar cuentas de usuario del sistema y los siguientes enteros para cuentas de usuario normales. Podemos añadir a la clase `Usuario` el siguiente fragmento de código:

```
bool get esSistema => _id < 1000;
```

En la clase `Usuario` no hay ninguna propiedad llamada `esSistema`. Cuando se llama a este getter se calcula el valor y se devuelve. En este ejemplo `true` o `false` según el `_id` sea menor que 1000 o mayor o igual que 1000 respectivamente.

Nota: Si el cálculo de una propiedad calculada necesita más de una línea de código, se usa la notación estándar en lugar de la notación flecha.

Setters

De forma similar, si se necesita cambiar el valor de una propiedad privada desde otro módulo, se pueden escribir **setters**.

Por ejemplo, si en la clase `Usuario` se necesita poder cambiar el valor de la propiedad `_nombre` desde otro módulo, se puede escribir el siguiente setter:

```
set nombre(String nuevoNombre) => _nombre = nuevoNombre;
```

Los setters en Dart comienzan con la palabra reservada `set`.

La forma de utilizar el setter es mediante la notación punto `.` habitual. Por ejemplo:

```
final usuario = Usuario(id: 45, nombre: 'Francisco');
usuario.nombre = 'Antonio';
print(usuario); // Usuario(id: 45, nombre: Antonio)
```

En este ejemplo, el setter asigna el valor que se pasa, directamente a la propiedad, pero también podría realizar cualquier proceso que se necesite, como por ejemplo, transformar el valor al formato interno que se almacena, comprobar la validez del valor que se quiere asignar, etc.

Como en el caso de los getters, si el cuerpo de un setter tiene más de una línea de código, entonces se debe usar la notación estándar.

Importante: No es obligatorio que el setter y el getter tengan el mismo nombre, pero es altamente recomendable, ya que de otra forma el código se vuelve muy confuso.

Tampoco es obligatorio que tengan el mismo nombre de la propiedad a la que acceden. Esto puede ser útil, si internamente se prefiere almacenar la propiedad en un formato y externamente se usa con otro.

Nota: Para una propiedad privada pueden tenerse getter y setter, getter pero no setter, setter pero no getter, o ninguno de los dos. Según interese en cada caso.

Nota: Obviamente, si se añade un setter para una propiedad, ésta no puede ser marcada como `final`. De igual forma, si una clase tiene propiedades que no son `final` entonces su constructor tampoco puede ser marcado como `const`.

Miembros de clase

Si se escribe la palabra reservada `static` delante del nombre de una propiedad o de un método, esa propiedad o método ya no pertenece a la instancia sino a la clase.

Nota: se suele hablar de propiedades o métodos estáticos, pero no es correcto. Lo correcto es hablar de propiedades o métodos **de clase**.

Un ejemplo sencillo es:

```
class UnaClase {
  static int unaPropiedad = 0;
  static void unMetodo() {
    print('Hola, Dart!');
  }
}
```

A las propiedades y métodos de clase se accede a través del nombre de la clase. En este ejemplo:

```
print(UnaClase.unaPropiedad); // 0
UnaClase.unMetodo(); // Hola, Dart!
```

Nota: Si se accede a una propiedad de clase desde dentro de la clase en la que se define, no es necesario usar el nombre de clase y el punto `.` para acceder a ella.

Importante: Como es habitual, no es necesario instanciar una clase para poder acceder a sus miembros (propiedades y métodos) de clase, ya que pertenecen a la clase y no a las instancias.

Las propiedades de clase pueden ser constantes y privadas. Para ello se usa la sintaxis que ya se ha visto para las propiedades de instancia.

Algunos ejemplos son:

```
static const unaConstante = '42';
```

declara una constante de clase. Es decir, su acceso no depende de que haya definida ninguna instancia, sino que se realiza a través de la clase, y además su valor no puede cambiarse durante la ejecución de la aplicación.

```
static const _idUserioAnonimo = 0;
static const _nombreUsuarioAnonimo = 'Anónimo';
```

declara dos propiedades de clase, que además son privadas y constantes, es decir, su valor no se puede cambiar una vez inicializadas y no pueden accederse desde fuera de la clase a menos que se haya proporcionado un getter.

Podríamos utilizar las dos propiedades anteriores en la clase `Usuario` para clarificar los valores por defecto del constructor.

La clase quedaría así:

```
class Usuario {
  static const _idUserioAnonimo = 0;
  static const _nombreUsuarioAnonimo = 'Anónimo';

  final int _id;
  String _nombre;

  int get id => _id;
  String get nombre { return _nombre; }
  set nombre(String nuevoNombre) => _nombre = nuevoNombre;
  bool get esSistema => _id < 1000;

  Usuario({int id = _idUserioAnonimo,
    String nombre = _nombreUsuarioAnonimo}):
    _id = id,
    _nombre = nombre;
  Usuario.anonimo() : this();
}
```

```

@override
String toString() {
    return 'Usuario(id: $_id, nombre: $_nombre)';
}
}

```

Importante: Las propiedades de clase, tienen **inicialización diferida**. Eso significa que Dart en realidad no calcula y asigna sus valores hasta que se usan la primera vez. Es una ayuda para obtener un mejor rendimiento en las aplicaciones móviles.

La utilidad de los métodos de clase es definir una utilidad o método auxiliar que esté asociado con la clase, pero no asociado con ninguna instancia en particular.

Por ejemplo se podría usar para construir objetos de la clase `Usuario` a partir de argumentos de entrada de tipo `String` :

```

static Usuario desdeStringV2({String idString = '0', String nombre = ''}) {
    int idNumero = int.parse(idString);
    return Usuario(id: idNumero, nombre: nombre);
}

```

Es muy parecido al constructor factoría estudiando anteriormente:

```

factory Usuario.desdeString({String idString = '0', String nombre = ''}) {
    int idNumero = int.parse(idString);
    return Usuario(id: idNumero, nombre: nombre);
}

```

Las diferencias entre métodos de clase o constructores factoría para construir objetos de la clase son:

- El constructor factoría solo puede devolver una instancia de la propia clase a la que pertenece, mientras que el método de clase puede devolver una instancia de cualquier clase definida.
- Los constructores factoría pueden no tener nombre, por lo que, desde el punto de vista de quien lo llama tiene el mismo aspecto de un constructor generativo. Los métodos de clase deben tener nombre siempre.
- Un constructor factoría puede ser marcado como `const` si todas sus propiedades son `final` mientras que un método de clase nunca puede ser marcado como `const` .

Mini ejercicios

1. Crea una clase llamada `Password` con una propiedad de tipo `String` llamada `valor` que sea privada.
2. Sobreescribe el método `toString` de la clase `Password` .
3. Añade un **método** a la clase `Password` llamado `longitudValida` que devuelva `true` solo si la longitud de `valor` es mayor o igual a 8 caracteres y `false` en caso contrario.

4. Sustituye el método `longitudValida` por una **propiedad calculada** equivalente, que también se llame `longitudValida`.
5. Añade a la clase `Password` la capacidad de cambiar el valor de la propiedad `valor`, pero que solo deje hacerlo cuando el nuevo valor tenga la longitud adecuada.
6. ➡ ¿Cómo podrías hacer para **garantizar** que todas las instancias que se creen de la clase `Password` tienen valores con la longitud adecuada? Implementa la solución propuesta.
7. Escribe una clase `Estudiante` con dos propiedades constantes de tipo `String` llamadas `nombre` y `apellidos` y una propiedad variable de tipo `double` llamada `nota`.

Añádele un constructor generativo personalizado que inicialice todas las propiedades y un método que formatee los objetos de la clase de forma adecuada para imprimir.

Usa la clase `Estudiante` para crear dos estudiantes con notas 9.5 y 8.5

8. ➡ Escribe una clase `Esfera`. Escribe un constructor marcado como `const` que reciba como parámetro con nombre un radio positivo. Añade getters para el volumen y el área de la esfera, pero no para el radio. No uses la biblioteca `dart:math` y la constante definida en ella `pi`, sino tu propia versión de clase de esa constante.

Usa la clase para calcular el volumen y área de una esfera de radio 12.

■ **Nota:** La fórmula del área de la esfera es: $4\pi r^2$ y la de su volumen es: $\frac{4}{3}\pi r^3$

9. ➡ Escribe una clase `Cilindro` que represente a un cilindro sólido (completamente cerrado). Escribe un constructor que reciba como parámetros un radio y una altura, ambos positivos. Añade getters para el volumen y el área del cilindro. Usa la constante `pi` de la biblioteca `dart:math`

Usa la clase para calcular el volumen y el área total de un cilindro de radio 2 y altura 5.

10. ¿Cómo mejoran el rendimiento de una aplicación móvil las propiedades de clase? Justifica tu respuesta.